LAB 7 REPORT

Fuzzing for Input Validation Bugs

Student Name: saketh narayanam

SRN: PES1UG24CS292

Section: E

Course: Software Engineering

Application: Photo Print Order

1. Introduction

In this lab, I used fuzz testing to check how the Photo Print Order application behaves when it receives different kinds of input. Instead of testing only a few values that I selected manually, I used the Hypothesis library to generate many inputs automatically. This helped me find cases that the original program did not handle properly, especially invalid values and unexpected data types.

2. Objectives

The main aim of this exercise was to understand how fuzz testing can be used to find input-validation problems. I tested all three functions in the application, identified the cases where validation was missing, corrected the code, and ran the same tests again to check whether the changes worked.

3. Tools and Environment

I carried out the experiment on a Windows system using Python 3.13.7. I used pytest 9.2.0 to execute the tests and Hypothesis 6.170.15 to generate test inputs automatically. The application used for testing was the Photo Print Order Python program.

4. Functions Tested

extra_print_charge(): This function calculates the extra amount a customer has to pay when the number of prints exceeds the included quantity. The calculation depends on the print size, finish type, and membership status.

order_valid(): This function checks whether an order follows the application's rules. Rush orders can contain up to 50 copies. Normal orders can contain up to 500 copies, and the uploaded file must not be larger than 25 MB.

dispatch_status(): This function checks the time available before dispatch and returns one of three results: on_time, tight, or missed. The cutoff depends on whether the order is framed and whether it is international.

5. Procedure Followed

I first kept a separate copy of the original application so that I could test it without changing the original code. I then created Hypothesis tests for the three functions. These tests generated different valid and invalid inputs, including negative numbers, unsupported values, and incorrect data types.

After running the tests against the original application, I checked the failures to understand what was going wrong. I then updated the application by adding input checks before the main calculations and conditions. Finally, I ran the same test file against the corrected application to confirm that the identified problems had been addressed.

6. Problems Found in the Original Application

The fuzz tests showed that the original program did not validate every input before using it. In extra_print_charge(), decimal or otherwise invalid copy values could be accepted, and unsupported finish values were treated like a default finish instead of being rejected. The function also did not check whether the membership value was actually a boolean.

In order_valid(), negative copy counts and negative file sizes were not rejected. The function also did not check the type of the file-size input or verify that the rush-order flag was a boolean.

In dispatch_status(), negative hours were accepted, and the function did not validate the type of the hours value. It also did not check whether the framed and international flags were booleans.

These issues could lead to incorrect results or allow invalid order information to pass through the application.

7. Changes Made to the Application

I added input validation to each of the three functions. The print-charge function now checks that the print size and finish are supported, that copies are a non-negative integer, and that membership status is a boolean. The order-validation function checks that file size is a non-negative number, copies are a non-negative integer, and rush status is a boolean. The dispatch function checks that the number of hours is a non-negative number and that both dispatch flags are booleans.

The corrected code raises TypeError when an argument has an incorrect data type and ValueError when an argument has an invalid value. The existing pricing rules, order limits, and dispatch cutoff logic were retained.

8. Test Results

When I ran the fuzz tests against the original application, 23 tests were collected. Out of these, 12 passed and 11 failed. The failures exposed the missing input checks described above. The original test run took 7.08 seconds.

After making the corrections, I ran the same set of 23 tests again. This time, all 23 tests passed, and there were no failures. The corrected test run took 9.96 seconds. This result shows that the corrected application satisfies the validation rules checked by the test suite.

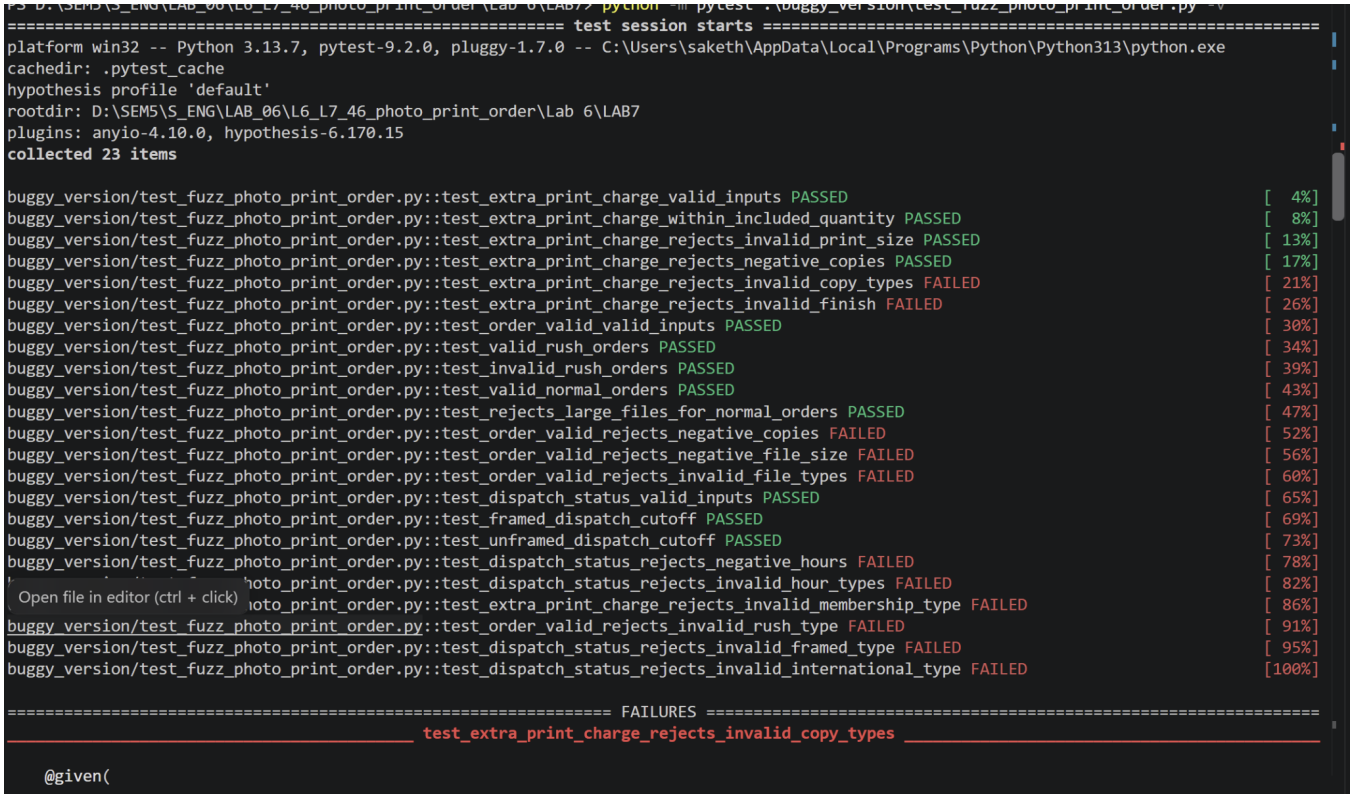
9. Conclusion

This lab helped me understand how property-based fuzz testing can reveal problems that may not appear when testing only a few normal examples. Hypothesis generated a range of inputs and helped identify missing validation in all three functions. After correcting the application and running the tests again, all 23 tests passed. The exercise showed the importance of checking inputs before using them in application logic.

10. Screenshot Evidence

The following screenshots show the test results before and after the application was corrected. The terminal output is included as evidence of the actual test runs.

A. Test run using the original application



The original version produced 11 failed tests and 12 passed tests.

B. Test run using the corrected application

```
============================================================ test session starts ============================================================
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\saketh\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: D:\SEM5\S_ENG\LAB_06\L6_L7_46_photo_print_order\Lab 6\LAB7
plugins: anyio-4.10.0, hypothesis-6.170.15
collected 23 items

test_fuzz_photo_print_order.py::test_extra_print_charge_valid_inputs PASSED                                                          [  4%]
test_fuzz_photo_print_order.py::test_extra_print_charge_within_included_quantity PASSED                                              [  8%]
test_fuzz_photo_print_order.py::test_extra_print_charge_rejects_invalid_print_size PASSED                                            [ 13%]
test_fuzz_photo_print_order.py::test_extra_print_charge_rejects_negative_copies PASSED                                               [ 17%]
test_fuzz_photo_print_order.py::test_extra_print_charge_rejects_invalid_copy_types PASSED                                            [ 21%]
test_fuzz_photo_print_order.py::test_extra_print_charge_rejects_invalid_finish PASSED                                                [ 26%]
test_fuzz_photo_print_order.py::test_order_valid_valid_inputs PASSED                                                                 [ 30%]
test_fuzz_photo_print_order.py::test_valid_rush_orders PASSED                                                                        [ 34%]
test_fuzz_photo_print_order.py::test_invalid_rush_orders PASSED                                                                      [ 39%]
test_fuzz_photo_print_order.py::test_valid_normal_orders PASSED                                                                      [ 43%]
test_fuzz_photo_print_order.py::test_rejects_large_files_for_normal_orders PASSED                                                    [ 47%]
test_fuzz_photo_print_order.py::test_order_valid_rejects_negative_copies PASSED                                                      [ 52%]
test_fuzz_photo_print_order.py::test_order_valid_rejects_negative_file_size PASSED                                                   [ 56%]
test_fuzz_photo_print_order.py::test_order_valid_rejects_invalid_file_types PASSED                                                   [ 60%]
test_fuzz_photo_print_order.py::test_dispatch_status_valid_inputs PASSED                                                             [ 65%]
test_fuzz_photo_print_order.py::test_framed_dispatch_cutoff PASSED                                                                   [ 69%]
test_fuzz_photo_print_order.py::test_unframed_dispatch_cutoff PASSED                                                                 [ 73%]
test_fuzz_photo_print_order.py::test_dispatch_status_rejects_negative_hours PASSED                                                   [ 78%]
test_fuzz_photo_print_order.py::test_dispatch_status_rejects_invalid_hour_types PASSED                                               [ 82%]
test_fuzz_photo_print_order.py::test_extra_print_charge_rejects_invalid_membership_type PASSED                                       [ 86%]
test_fuzz_photo_print_order.py::test_order_valid_rejects_invalid_rush_type PASSED                                                    [ 91%]
test_fuzz_photo_print_order.py::test_dispatch_status_rejects_invalid_framed_type PASSED                                              [ 95%]
test_fuzz_photo_print_order.py::test_dispatch_status_rejects_invalid_international_type PASSED                                       [100%]

============================================================ 23 passed in 9.96s =============================================================
```

The corrected version completed with all 23 tests passing.